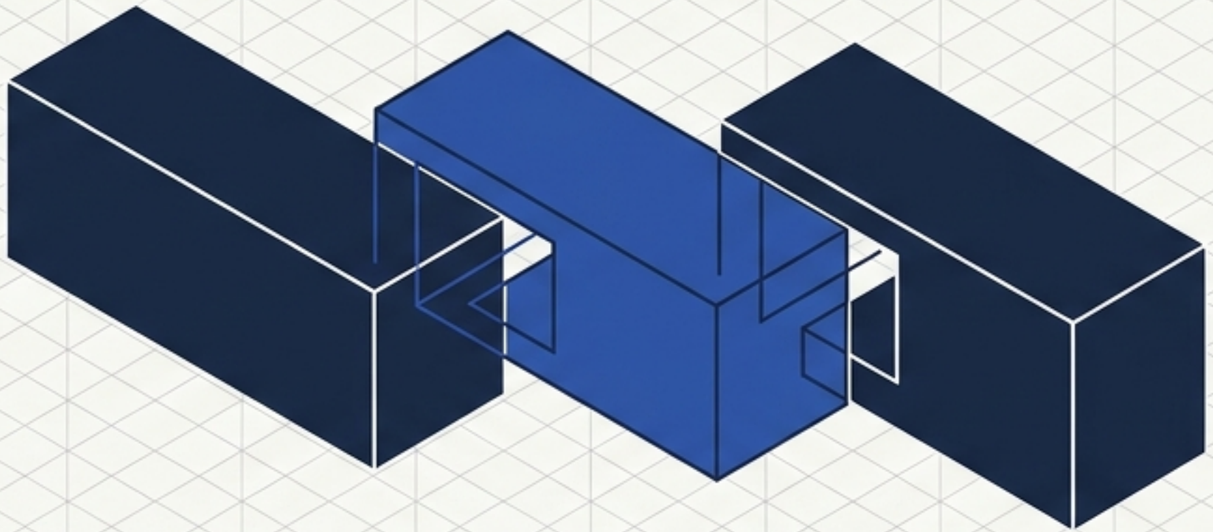
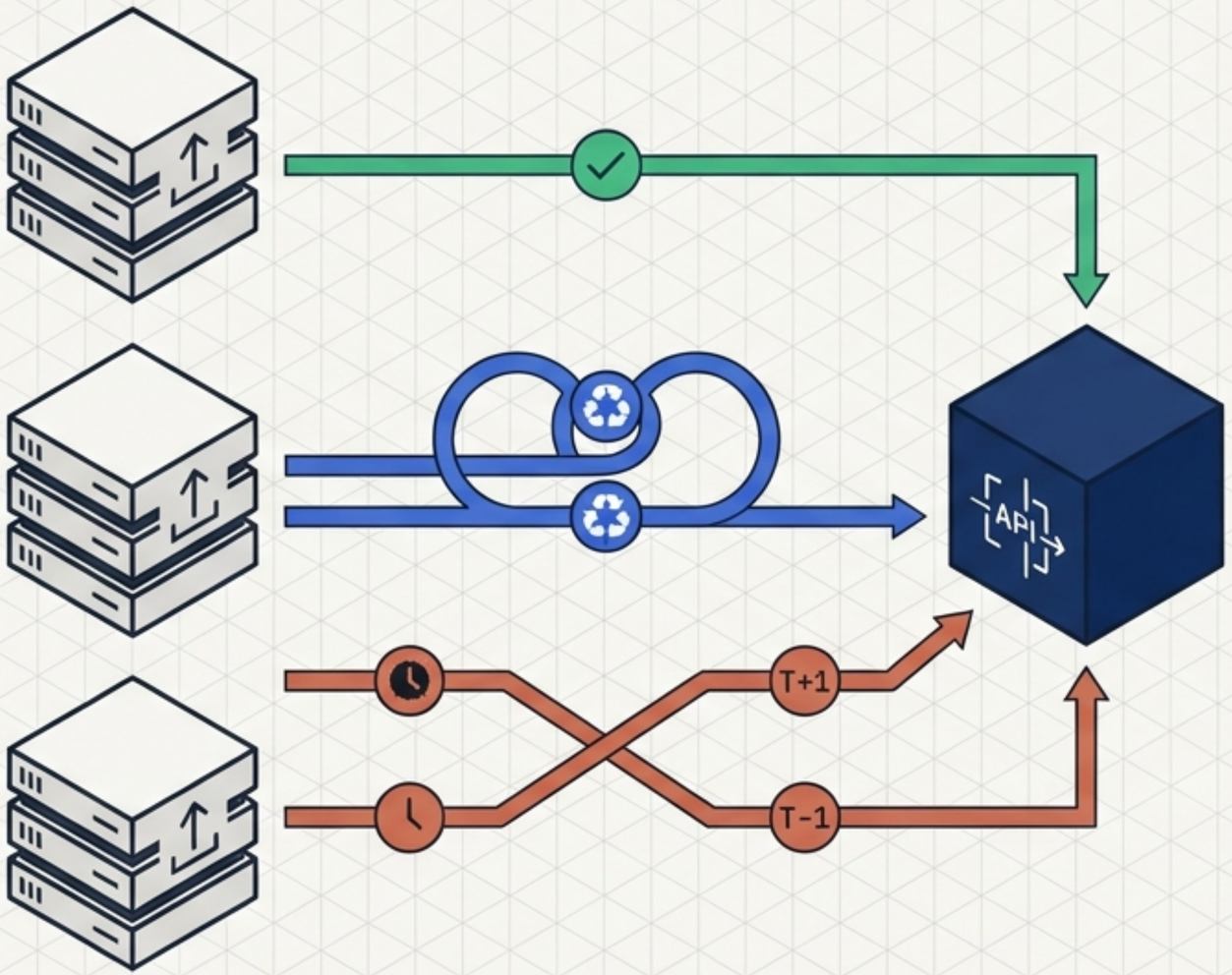


Architecture & Implementation: Event Ledger API

Building an idempotent, out-of-order tolerant financial ledger.

`TARGET: TECHNICAL_REVIEW | SPEC: 001-EVENT-LEDGER-API`

The Reality of Distributed Financial Events



At-Least-Once Semantics

Network failures cause upstream systems to retry blindly, resulting in duplicate delivery.

Time Skew

Upstream clocks and queues are unsynchronized; older events frequently arrive after newer ones.

Eventual Delivery

The API must act as the ultimate source of truth, enforcing order, uniqueness, and financial accuracy out of upstream chaos.

Bounding the Problem Space

Functional Rules

- Idempotent processing
- Strict chronological retrieval
- Accurate balance derivation

Technical Constraints

- Python 3.11+ environment
- No external DB dependencies
- Embedded local storage only

Financial Integrity

- Exact decimal arithmetic (no floating-point drift)
- Strict multi-currency isolation

Developer Experience

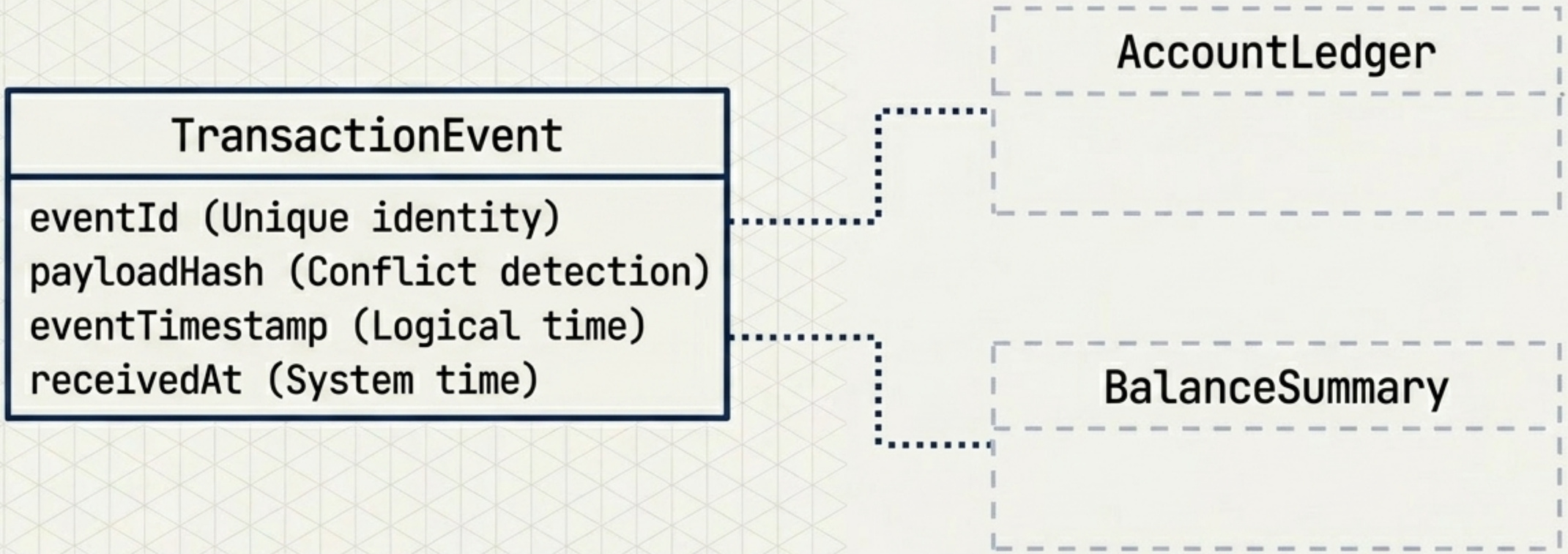
- Single-command local execution
- Comprehensive test coverage

Foundational Stack Decisions

Dimension	Chosen ✅	Rejected ❌
Language & Framework	Python + FastAPI / Pydantic v2. Provides strong typing and OpenAPI-native contracts while remaining compact.	Java/Spring Boot. Introduces excessive boilerplate for a strictly local exercise.
Storage Engine	Embedded SQLite. ACID compliant, enforces unique constraints natively, requires zero setup.	Pure in-memory dictionaries. Lacks durability and fails to demonstrate query index proofs.
Verification	Pytest with in-memory SQLite isolation. Ensures repeatable, fast test execution independent of runtime state.	(Pure unit mocks). Fails to test concurrent database constraints realistically.

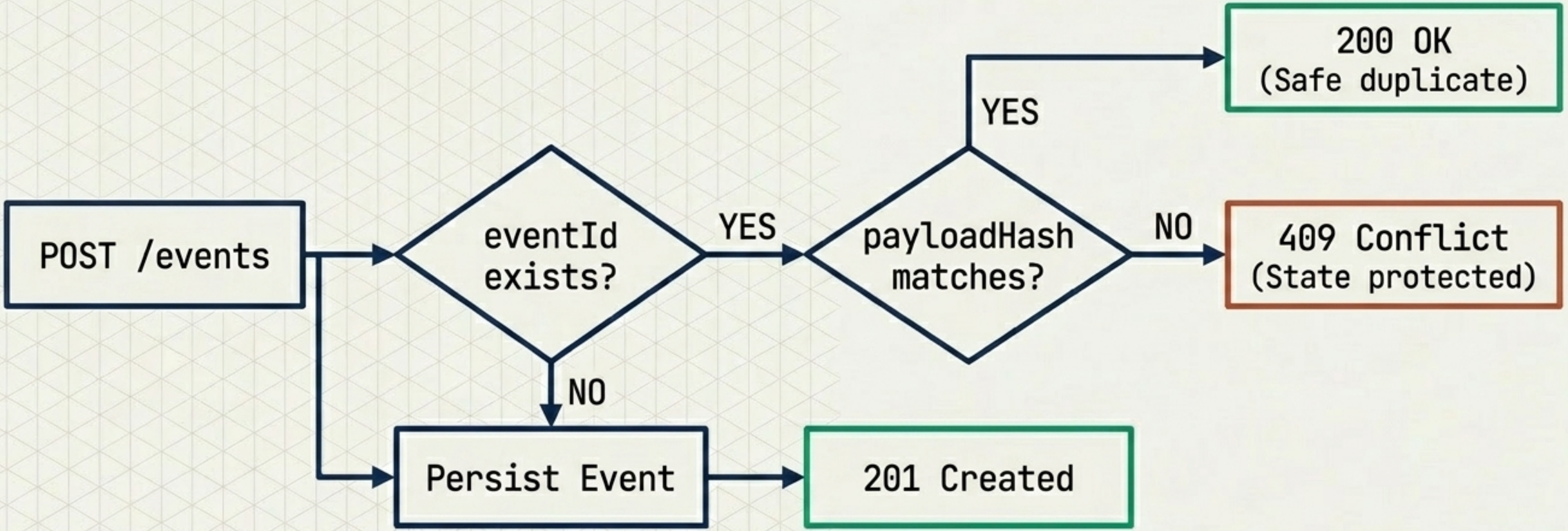
Immutable Single Source of Truth

State is purely a derivation of events. There is no mutable “Account” table. Accepted events have no update or delete transitions.



Canonical Hashing for Safe Replays

How do we distinguish a harmless network retry from a conflicting attempt to reuse an ID? The API computes a canonical hash of the balance-affecting payload upon receipt.



Decoupling Arrival Time from Logical Time

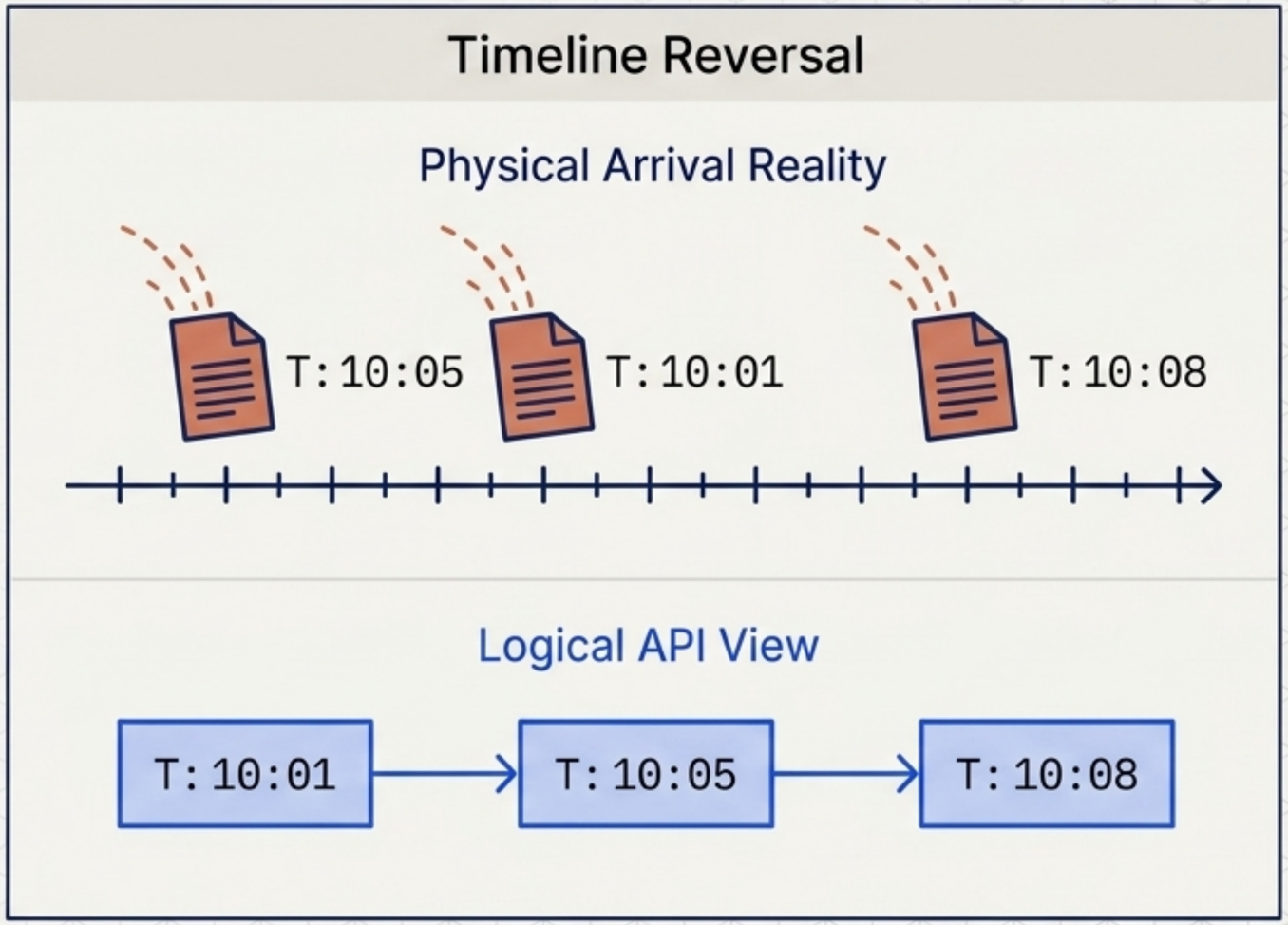
The Mechanism

The system explicitly ignores the internal `receivedAt` system time.

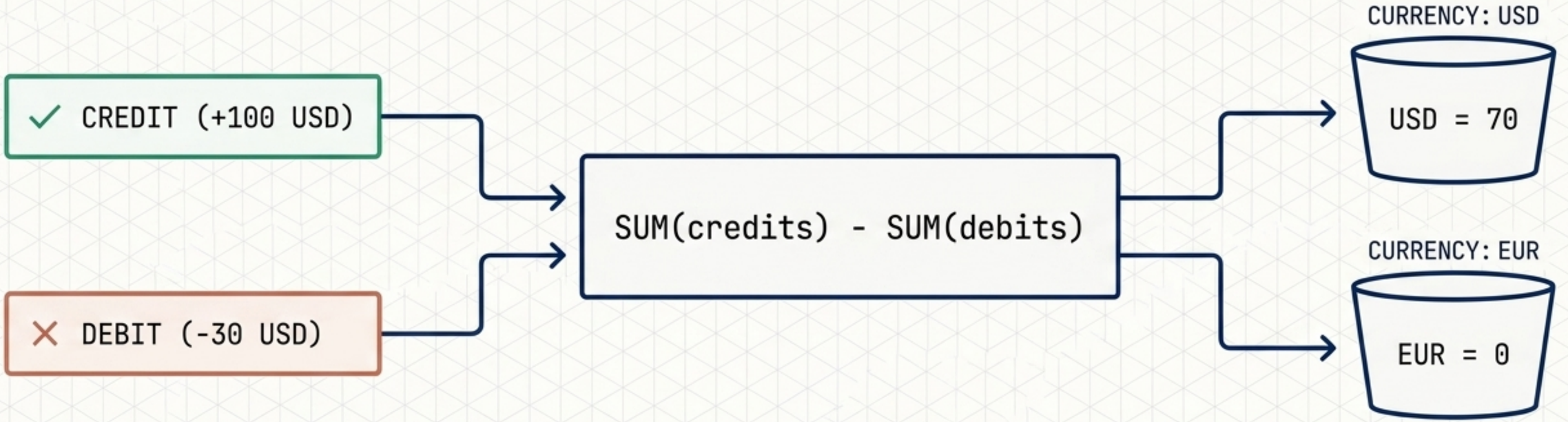
Account history is queried using a strict index on (`accountId`, `eventTimestamp`, `eventId`).

Deterministic Tie-Breaking

`eventId` serves as a stable secondary sort key, ensuring stable responses for identical timestamps.



Event-Sourced Balance Computation



Derivation on Read

Balances are never stored. They are computed dynamically from accepted unique events. This eliminates the risk of balance-drift, race conditions, or double-counting during concurrent writes.

Currency Isolation

Balances are strictly grouped by the currency field. The system avoids mathematically netting incompatible currencies, fulfilling strict financial logic boundaries.

Exact Decimal Arithmetic

Financial amounts must never be subjected to binary floating-point rounding.
Amounts <= 0 are strictly rejected.

Floating Point Math	Exact Decimal Math
`0.1 + 0.2 = 0.30000000000000004`	`Decimal('0.1') + Decimal('0.2') = Decimal('0.3')`
Inherent binary drift risk.	Pydantic casts JSON numbers to pure Python Decimal types.

Graceful Degradation & State Protection

`422 Unprocessable Entity`	Triggered by malformed timestamps, unknown event types, or negative amounts. (Zero ledger mutation).
`409 Conflict`	Triggered by reusing an accepted eventId with a different payload. (Zero ledger mutation).
`404 Not Found`	Triggered by a direct lookup of an unknown ID.
`400 Bad Request`	Triggered by missing the required accountId query parameter.

Standardized Schema: All errors return a uniform `ErrorResponse` (`error.code`, `error.message`, `error.details`) for strict machine readability.

The External API Contract

`POST /events`

Submit transaction (Idempotent ingest)

`GET /events/{id}`

Retrieve single event (Audit)

`GET /events?account={id}`

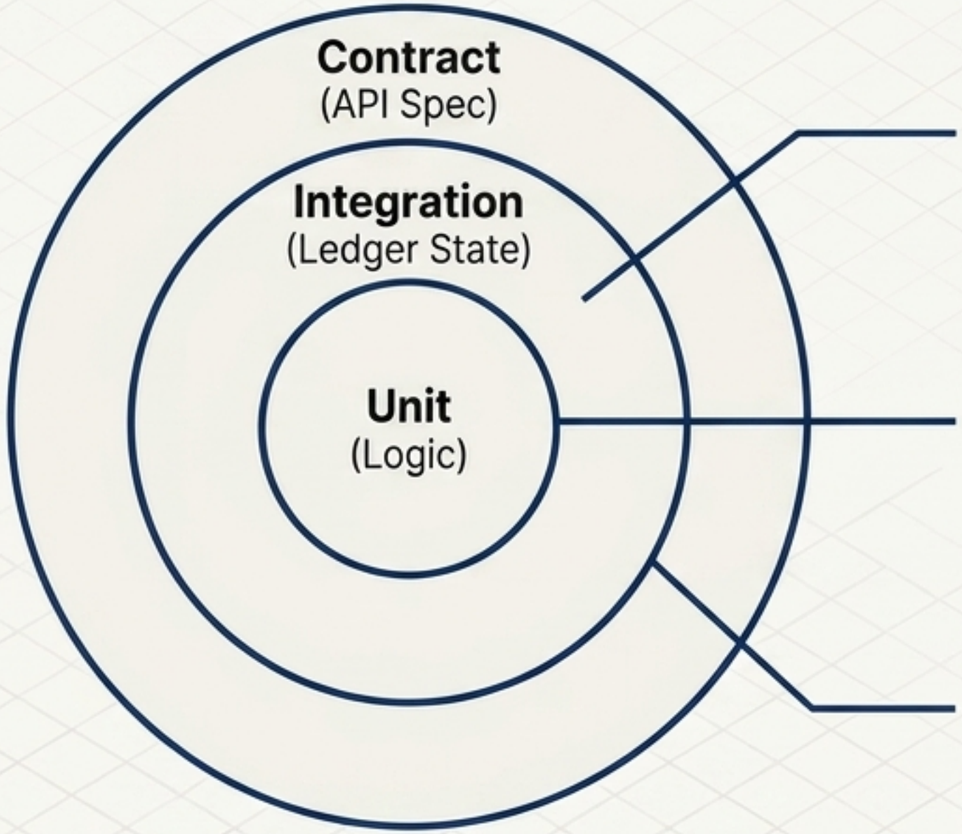
List history (Chronological reconciliation)

`GET /accounts/{id}/balance`

Get computed net state (Grouped by currency)

Spec Alignment: Fully documented via natively generated OpenAPI JSON/YAML (`/docs`).

Rigorous Local Verification



Unit Tests

Verifies exact decimal math and canonical payload hashing.

Integration Tests

Verifies out-of-order listing stability, multi-currency balance accuracy, and concurrent exact-duplicate handling using isolated in-memory SQLite fixtures.

Contract Tests

Ensures endpoints match predefined OpenAPI specs natively.

The Guarantee: **100%** of exact duplicates produce **exactly one ledger effect** in the automated test suite.

Zero-Friction Local Execution

The Goal: Start the application and run all tests in under 10 minutes from a clean checkout, without external DB provisioning.

```
$ source .venv/bin/activate
$ uvicorn src.event_ledger_api.main:app --reload-
  dir src
$ pytest
```

Anticipating Friction

Built-in `bootstrap-venv.sh` script explicitly bypasses macOS/iCloud Desktop dataless file blocking issues, preventing Python `ModuleNotFoundError` hangs.

Project Constitution Delivered

✅x] **Ledger Integrity Maintained**: Append-only persistence enforced.

✅x] **Idempotency Guaranteed**: Solved via canonical payload hashing.

✅x] **Time Skew Resolved**: Strict chronological ordering enforced on read.

✅x] **Financial Accuracy**: Zero floating-point drift.

✅x] **Frictionless Review**: Zero external dependencies; local execution confirmed.

Status: A resilient, extensible ledger foundation built to spec, ready for technical review.
API Docs accessible locally at `http://127.0.0.1:8000/docs`